

Scalability & Future Plan Template

LaunchPad AI — Project Documentation

How LaunchPad AI's current architecture can scale to handle more users, and what's planned for future releases.

Scalability

Aspect	Details
Current Architecture	A single Flask process serving all requests, backed by a local SQLite database (instance/launchpad.db), with synchronous calls out to the Gemini 1.5 Flash API on every analysis request, and one auto-launched Ngrok tunnel exposing the app publicly
Scalability Approach	Migrate from SQLite to PostgreSQL to support concurrent writes safely; replace the single-process Ngrok setup with a proper WSGI server (Gunicorn) behind Nginx, or move to a managed host (Render, Railway, or AWS); move Gemini API calls off the request thread into an async task queue (Celery/RQ) so slow AI responses don't block other users; add response caching for repeated/similar startup descriptions to reduce API calls
Expected Growth / Load	Current setup comfortably handles the single-user demo/evaluation use case (a handful of concurrent requests). With the changes above, the same codebase can reasonably scale to a few hundred daily active users before further infrastructure work (load balancing, read replicas) would be needed

Future Plan

S. No	Planned Feature / Enhancement	Target Timeline	Priority
1	User authentication & multi-user support (separate report history per user)	Next 1–2 months	High
2	Export generated reports to PDF / Excel	Next 2–4 weeks	Medium
3	Visual charts for viability/innovation score trends over time	Next month	Medium
4	Multi-LLM support (fallback to another model if Gemini is unavailable)	Next 2–3 months	Low